

Real-Time Video Telemetry Pipeline

Overview

I have implemented a real-time Quality of Service (QoS) monitoring pipeline for HLS video streaming. Events emitted by video players (buffer stalls, bitrate changes, heartbeats) are collected, streamed through Kafka, processed by Spark Structured Streaming, persisted in ClickHouse, and visualised in Grafana — all running as Docker containers.

1.1 Architecture Overview

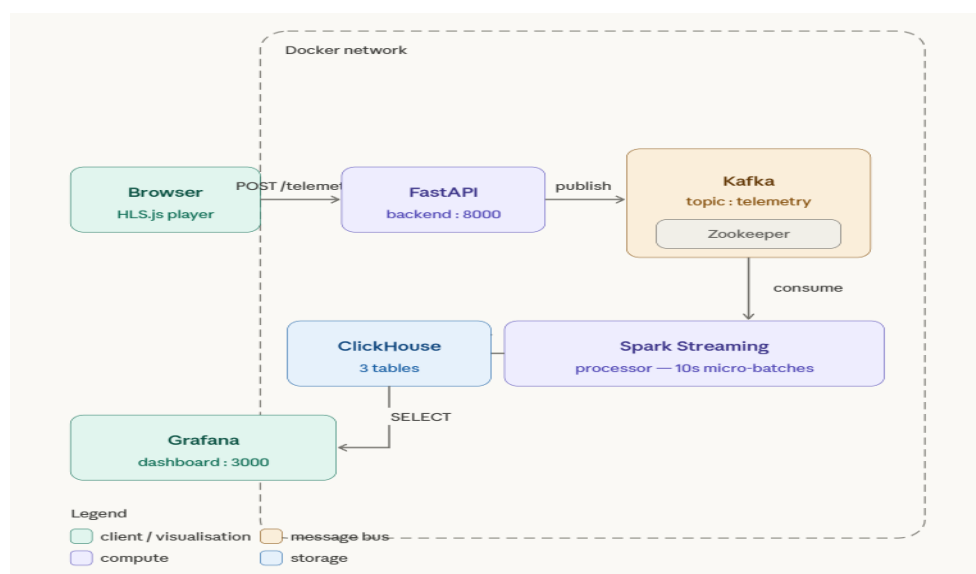
The data flows through five layers:

- Browser (HLS.js player) emits telemetry events via fetch() to a FastAPI endpoint
- FastAPI publishes each event as a JSON message to a Kafka topic
- Apache Spark consumes the Kafka topic in 10-second micro-batches
- Spark writes raw events and derived metrics to ClickHouse tables
- Grafana queries ClickHouse and renders live dashboards

1.2 Key Features

- Raw event ingestion: every player event stored in `telemetry_events`
- Buffer duration calculation: `BUFFER_END - BUFFER_START` per viewer/session
- State expiry: sessions buffering > 5 minutes auto-flushed to `buffer_timeouts`
- Orphan event handling: `BUFFER_END` with no matching start recorded as timeout
- Grafana dashboard: events/min, avg bitrate, buffer stalls, per-user analytics

End-to-end pipeline flow:



2. Docker Setup

All services are defined in a single `docker-compose.yml`. The stack has six containers:

Container	Purpose
zookeeper	Kafka coordination service
kafka	Message broker (topic: telemetry)
clickhouse	Columnar database for analytics
backend	FastAPI ingestion endpoint
processor	Spark Structured Streaming job
grafana	Dashboard visualisation

3. ClickHouse Setup

3.1 Credentials

Setting	Value
Host	clickhouse
Port	9000 (native TCP)
User	default
Password	clickhouse
Database	default

3.2 init.sql — Schema Definition

The file is mounted at /docker-entrpoint-initdb.d/init.sql and executed automatically on first container start against a fresh volume.

1. telemetry_events

Stores every raw event received from the player.

```
CREATE TABLE IF NOT EXISTS telemetry_events (
  event_id      String,
  timestamp     Int64,          -- epoch ms from browser
  viewer_id     String,
  session_id    String,
  event_type    String,        -- HEARTBEAT | BUFFER_START | BUFFER_END |
  QUALITY_CHANGE
  video_position Float64,
  bitrate       Int32,          -- kbps
  processed_at  DateTime
) ENGINE = MergeTree()
ORDER BY (viewer_id, session_id, timestamp);
```

2. buffer_durations

Stores one row per completed buffer stall (BUFFER_END - BUFFER_START).

```
CREATE TABLE IF NOT EXISTS buffer_durations (  
  viewer_id      String,  
  session_id     String,  
  buffer_start_ts Int64,  
  buffer_end_ts  Int64,  
  duration_ms    Int64,  
  calculated_at  DateTime  
) ENGINE = MergeTree()  
ORDER BY (viewer_id, session_id, buffer_start_ts);
```

3. buffer_timeouts

Stores buffer sessions that were never closed (timeout) or END events with no matching START (orphan_end).

```
CREATE TABLE IF NOT EXISTS buffer_timeouts (  
  viewer_id      String,  
  session_id     String,  
  reason         String,      -- 'timeout' | 'orphan_end'  
  event_type     String,  
  event_ts       Int64,  
  start_ts       Int64,  
  elapsed_ms     Int64,  
  recorded_at    DateTime  
) ENGINE = MergeTree()  
ORDER BY (viewer_id, session_id, recorded_at);
```

4. Kafka Setup

Configuration

```
Bootstrap server : kafka:9092  
Topic            : telemetry  
Consumer group   : processor-group  
Max offsets/batch: 1000
```

5. Backend — FastAPI Ingestion Service

Endpoint

```
POST http://localhost:8000/telemetry  
Content-Type: application/json
```

6. Frontend — Telemetry Collector (HLS.js)

6.1 Event Types Emitted

Event Type	Trigger
HEARTBEAT	Every 5 seconds while page is open
BUFFER_START	On video 'waiting' event (first occurrence)
BUFFER_END	On video 'playing' event after a buffer stall
QUALITY_CHANGE	On Hls.Events.LEVEL_SWITCHED

6.2 Bitrate Capture

Bitrate is cached on MANIFEST_PARSED and LEVEL_SWITCHED events rather than read on demand. This avoids NaN values when `hls.currentLevel` is -1 during buffering (which `JSON.stringify` silently drops). The `lastBitrate` field ensures all events — including heartbeats — always carry a valid integer.

6.3 Payload Shape

```
{
  "event_id"      : "abc123",
  "timestamp"     : 1774995333309,    // Date.now() - epoch ms
  "viewer_id"     : "viewer_460",
  "session_id"    : "wlwjyv",
  "event_type"    : "HEARTBEAT",
  "video_position": 23.823,
  "bitrate"       : 1500               // kbps, always an integer
}
```

7. Spark Structured Streaming Processor

7.1 Stream Configuration

```
Trigger      : processingTime = 10 seconds
Output mode   : append
Checkpoint   : /checkpoints/telemetry_events
Max offsets  : 1000 per trigger
```

7.2 ClickHouse Connection with Retry

The processor retries the ClickHouse connection up to 10 times, probing all three tables with `SELECT 1 FROM <table> LIMIT 0` on each attempt. A bare TCP connection check is not sufficient because ClickHouse accepts connections a few seconds before finishing DDL from `init.sql`.

7.3 write_batch() Logic

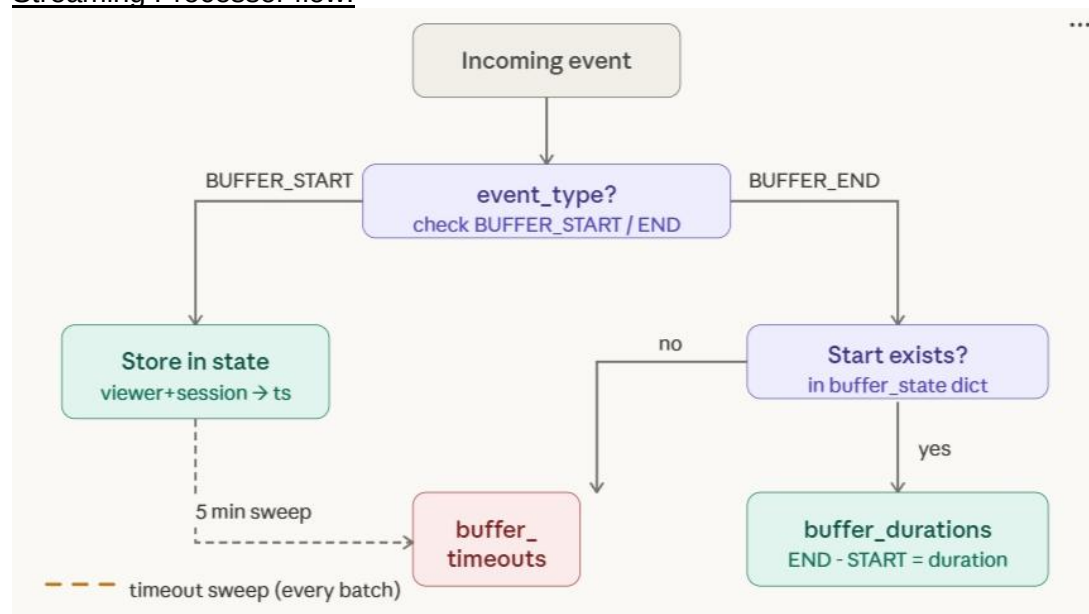
Each micro-batch executes five steps in order:

- Step 1 — Timeout sweep: scan `buffer_state` for entries older than 5 minutes, evict them, stage rows for `buffer_timeouts`
- Step 2 — Raw insert: write all events in the batch to `telemetry_events`
- Step 3 — State transitions: sort rows by timestamp, process `BUFFER_START` / `BUFFER_END` events
- Step 4 — Duration insert: write completed `BUFFER_END - BUFFER_START` pairs to `buffer_durations`
- Step 5 — Timeout insert: write timeout and orphan_end rows to `buffer_timeouts`

7.4 State Machine

Event / Condition	Action
BUFFER_START received	Store (viewer_id, session_id) -> timestamp in buffer_state dict
BUFFER_END received, start exists	Pop start from dict, calculate duration_ms, write to buffer_durations
BUFFER_END received, no start	Write to buffer_timeouts with reason = orphan_end
Entry in state > 5 minutes	Evict from dict, write to buffer_timeouts with reason = timeout

Streaming Processor flow:



Output:

```
client-1 | 2026/07/05 07:49:16 [error] 31831: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.18.0.1, server: localhost, request: "GET /favicon.ico HTTP/
1.1", host: "localhost:3000", referer: "http://localhost:3000/"
backend-1 | Received: {'event_id': '5vj3k', 'timestamp': 1783237759069, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'HEARTBEAT', 'video_position': 268.088, 'bitrate': 2149}
backend-1 | INFO: 172.18.0.1:54646 - "POST /telemetry HTTP/1.1" 200 OK
backend-1 | Received: {'event_id': 'lv4e93', 'timestamp': 1783237760393, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'QUALITY_CHANGE', 'video_position': 268.088, 'level': 4, 'bitrate': 6222}
backend-1 | INFO: 172.18.0.1:54646 - "POST /telemetry HTTP/1.1" 200 OK
backend-1 | Received: {'event_id': '71fbj', 'timestamp': 1783237760394, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'BUFFER_END', 'video_position': 268.088, 'bitrate': 6222}
backend-1 | INFO: 172.18.0.1:54646 - "POST /telemetry HTTP/1.1" 200 OK
processor-1 | [4] raw events -> 3 rows
processor-1 | Batch progress: 4 input rows | state size: 1
backend-1 | Received: {'event_id': 'sejag1', 'timestamp': 1783237762431, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'QUALITY_CHANGE', 'video_position': 270.056, 'level': 2, 'bitrate': 836}
backend-1 | INFO: 172.18.0.1:54646 - "POST /telemetry HTTP/1.1" 200 OK
backend-1 | Received: {'event_id': 'bzj3it', 'timestamp': 1783237764003, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'BUFFER_START', 'video_position': 451.728, 'bitrate': 836}
backend-1 | INFO: 172.18.0.1:54646 - "POST /telemetry HTTP/1.1" 200 OK
backend-1 | Received: {'event_id': 'x4yz1', 'timestamp': 1783237764069, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'HEARTBEAT', 'video_position': 451.728, 'bitrate': 836}
backend-1 | INFO: 172.18.0.1:54646 - "POST /telemetry HTTP/1.1" 200 OK
backend-1 | Received: {'event_id': 'x7n19s', 'timestamp': 1783237766782, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'QUALITY_CHANGE', 'video_position': 451.728, 'level': 3, 'bitrate': 2149}
backend-1 | INFO: 172.18.0.1:54646 - "POST /telemetry HTTP/1.1" 200 OK
backend-1 | Received: {'event_id': '96lo5q', 'timestamp': 1783237766782, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'BUFFER_END', 'video_position': 451.728, 'bitrate': 2149}
backend-1 | INFO: 172.18.0.1:56590 - "POST /telemetry HTTP/1.1" 200 OK
backend-1 | Received: {'event_id': 'lg5j67', 'timestamp': 1783237769067, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'HEARTBEAT', 'video_position': 453.403, 'bitrate': 2149}
backend-1 | INFO: 172.18.0.1:56590 - "POST /telemetry HTTP/1.1" 200 OK
processor-1 | [5] raw events -> 8 rows
processor-1 | [5] durations -> 2 rows
processor-1 | Batch progress: 9 input rows | state size: 0
backend-1 | Received: {'event_id': 'kyw2r', 'timestamp': 1783237774066, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'HEARTBEAT', 'video_position': 453.403, 'bitrate': 2149}
backend-1 | INFO: 172.18.0.1:56590 - "POST /telemetry HTTP/1.1" 200 OK
backend-1 | Received: {'event_id': 'nqk77', 'timestamp': 1783237779068, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'HEARTBEAT', 'video_position': 453.403, 'bitrate': 2149}
backend-1 | INFO: 172.18.0.1:56590 - "POST /telemetry HTTP/1.1" 200 OK
processor-1 | [6] raw events -> 2 rows
processor-1 | Batch progress: 3 input rows | state size: 0
backend-1 | Received: {'event_id': 'rkjape', 'timestamp': 1783237784070, 'viewer_id': 'viewer_197', 'session_id': 's0ecum', 'event_type': 'HEARTBEAT', 'video_position': 453.403, 'bitrate': 2149}
backend-1 | INFO: 172.18.0.1:56590 - "POST /telemetry HTTP/1.1" 200 OK
```

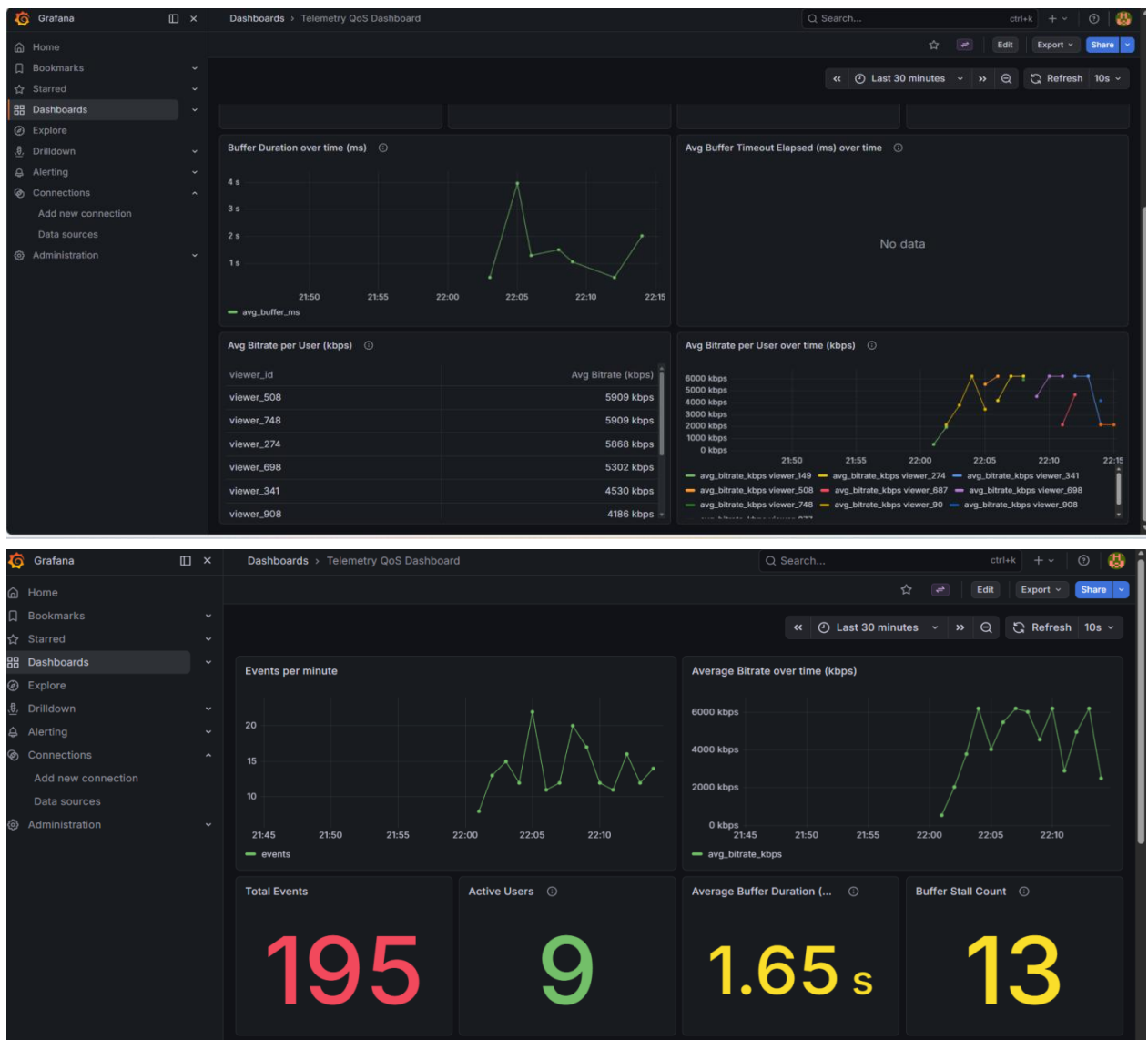
8. Grafana Dashboard

8.1 Datasource

Plugin : grafana-clickhouse-datasource
UID : clickhouse-ds
Host : clickhouse:9000
User : default
Password: clickhouse

8.2 Dashboard Panels

Panel	Query Summary
Events per minute	Time series – COUNT(*) from telemetry_events grouped by minute
Avg Bitrate over time	Time series – AVG(bitrate) where bitrate > 0
Total Events	Stat – COUNT(*) in selected time range
Active Users	Stat – COUNT(DISTINCT viewer_id) from HEARTBEAT events
Avg Buffer Duration	Stat – AVG(duration_ms) from buffer_durations
Buffer Stall Count	Stat – COUNT(*) from buffer_durations
Buffer Duration over time	Time series – AVG(duration_ms) per minute
Avg Timeout Elapsed	Time series – AVG(elapsed_ms) from buffer_timeouts
Avg Bitrate per User	Table – per viewer_id average
Bitrate per User over time	Time series – per viewer trend



8.3 Importing the Dashboard

1. Open Grafana at `http://localhost:3000`
2. Default credentials: `admin / admin`
3. Go to Dashboards -> Import
4. Upload `dashboard.json`
5. Select `clickhouse-ds` as the datasource

9. Project Structure

```
.
├── docker-compose.yml
├── init.sql           # ClickHouse DDL - all 3 tables
├── backend/
│   ├── Dockerfile
│   └── main.py        # FastAPI + KafkaProducer
├── processor/
│   ├── Dockerfile
│   └── processor.py   # Spark Structured Streaming job
├── frontend/
│   └── telemetry.js   # TelemetryCollector class
└── grafana/
    └── dashboard.json # Grafana dashboard definition
```